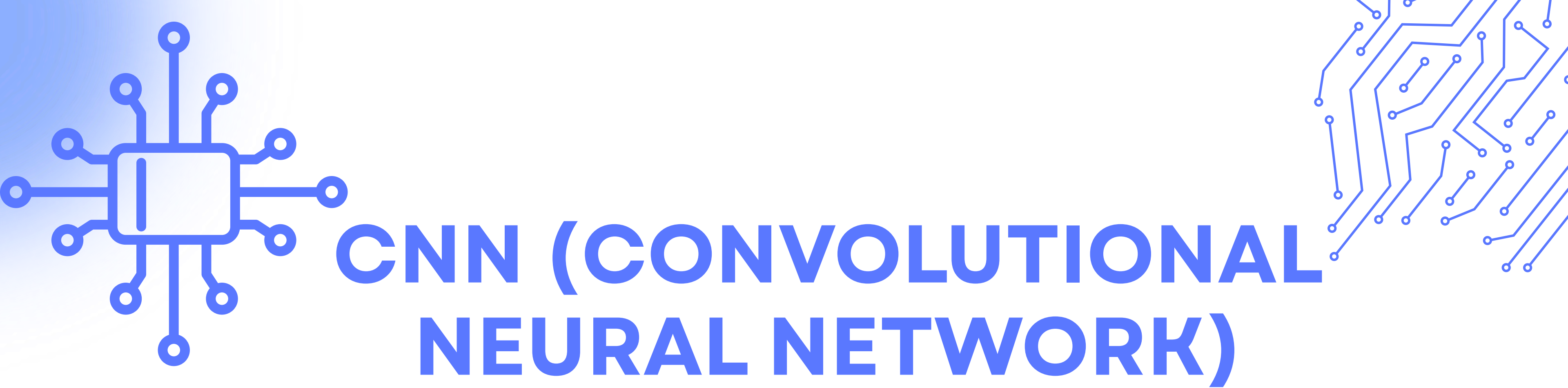
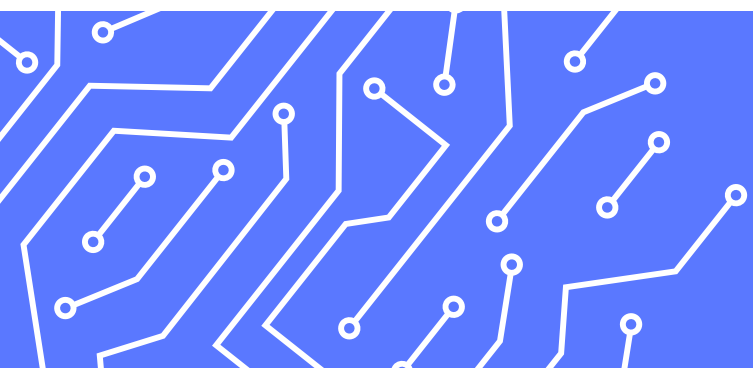
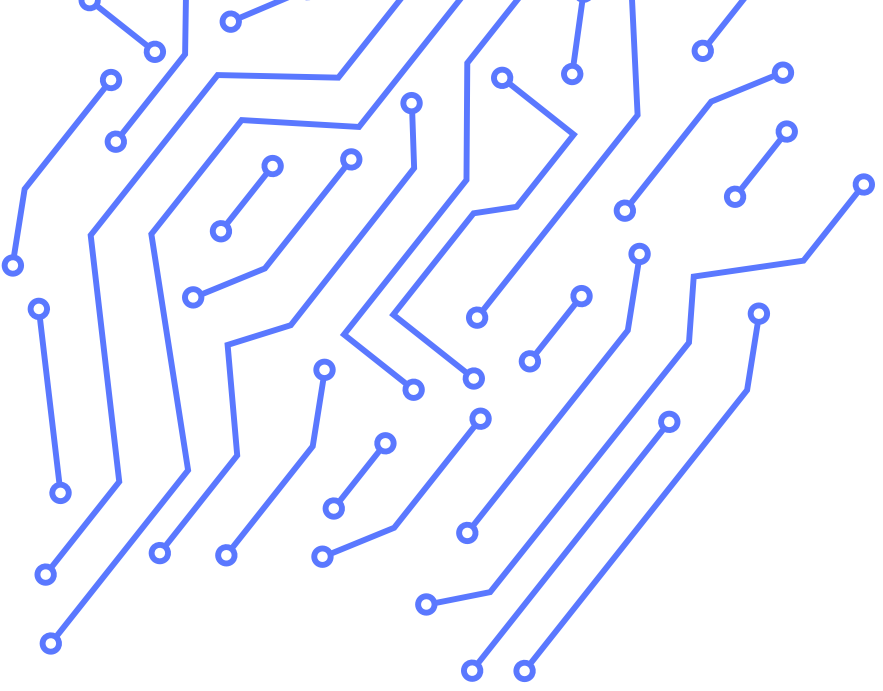




CNN (CONVOLUTIONAL NEURAL NETWORK)

**MADE BY:
ALI RAAFAT**





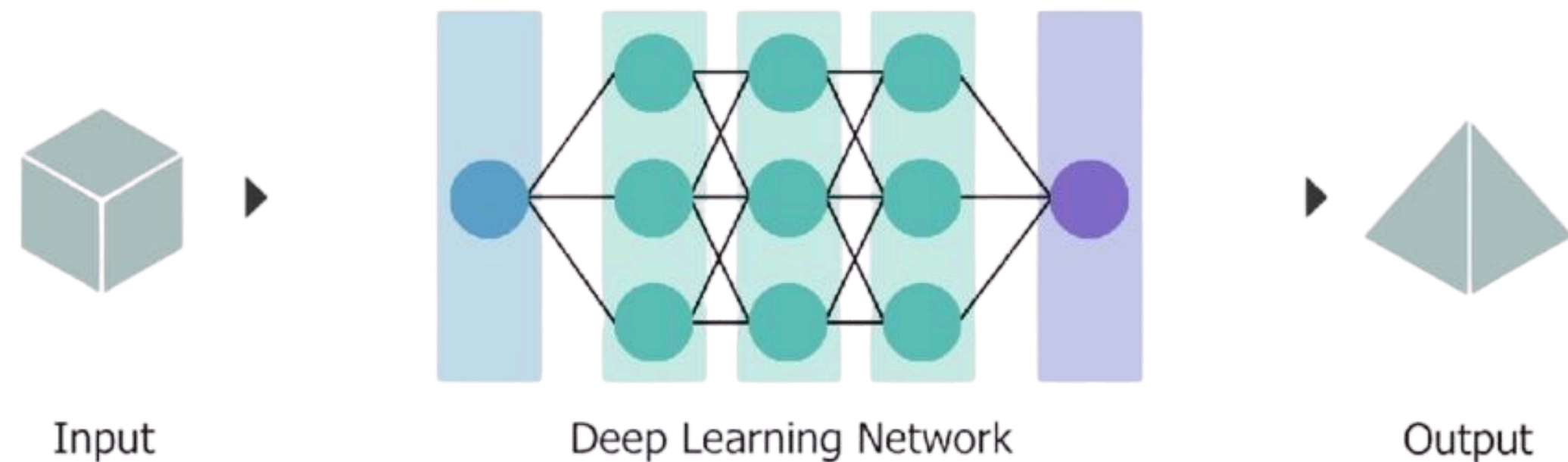
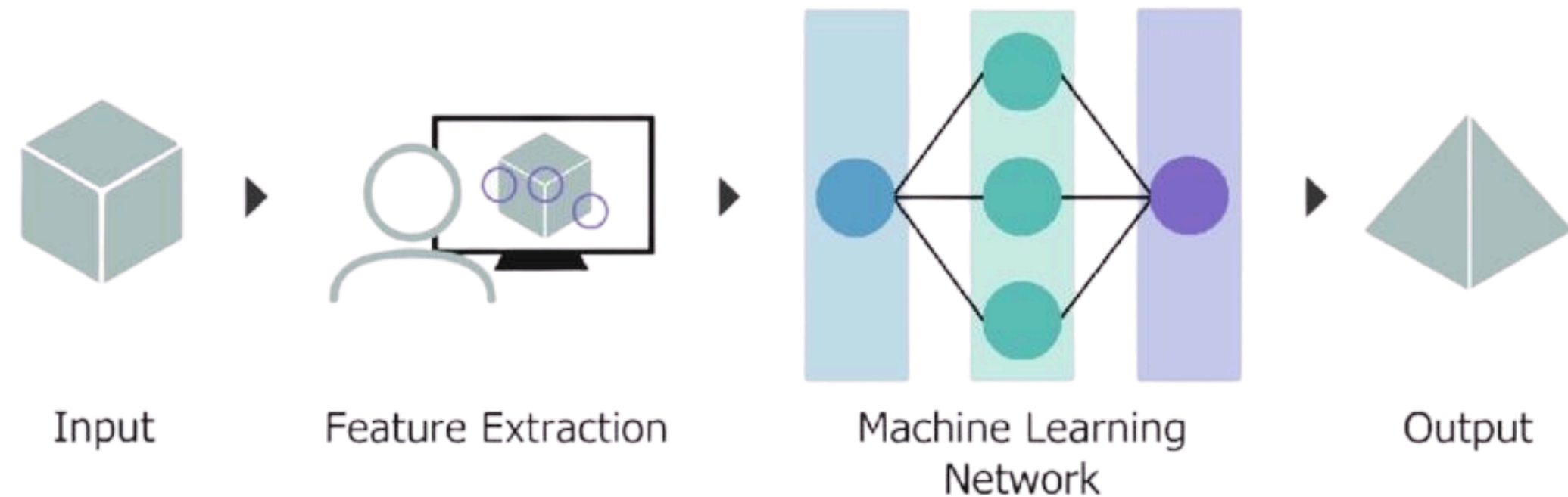
Section 1

How Images Were Processed Before CNNs



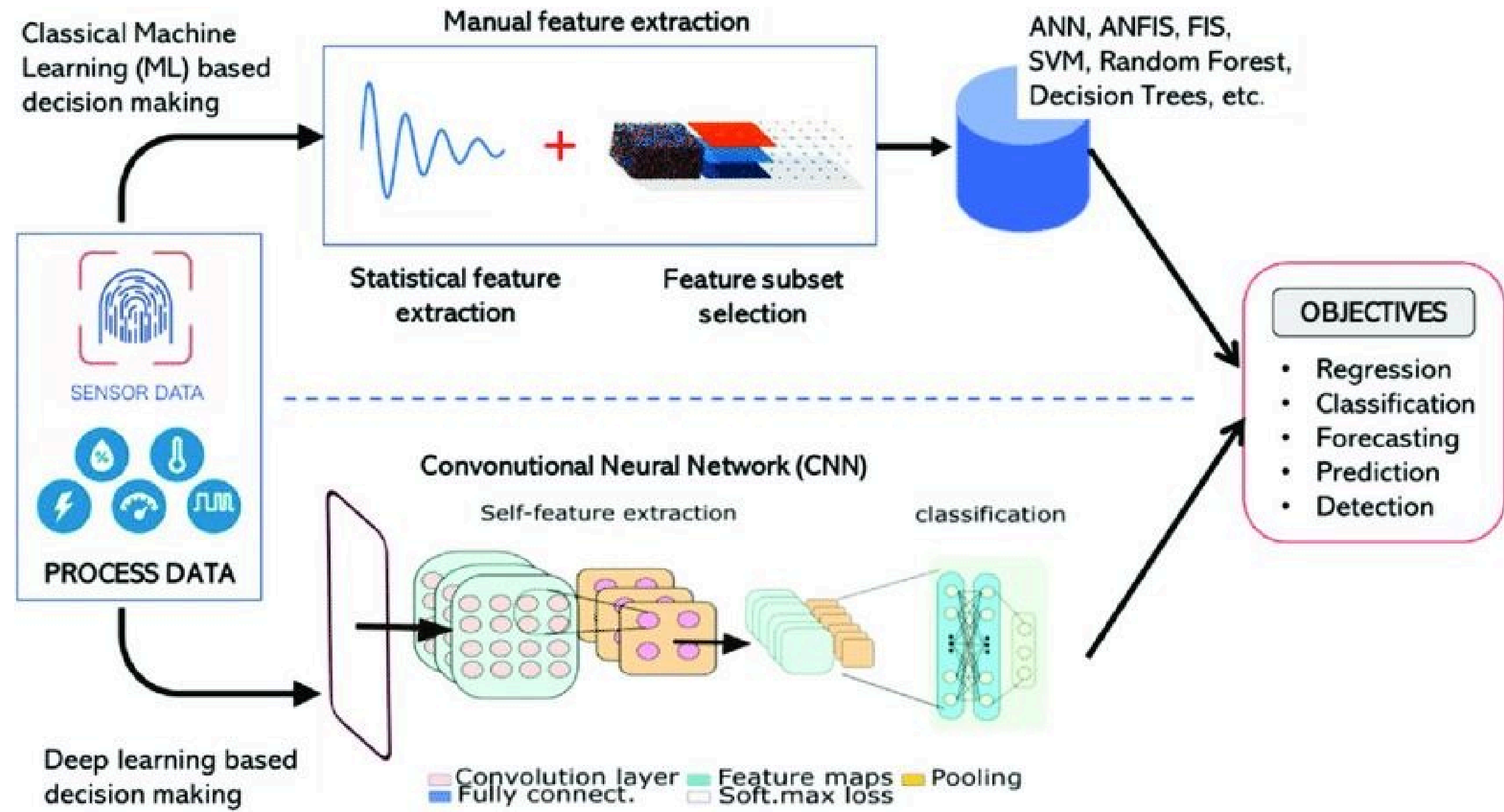
MANUAL FEATURE ENGINEERING

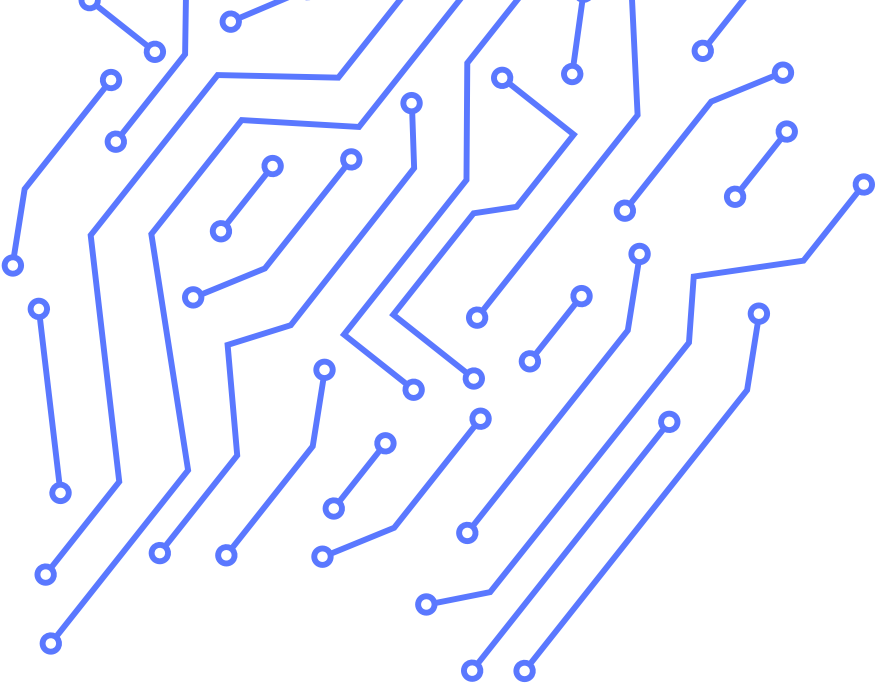
Before CNNs, computers couldn't learn what to look for in an image on their own. Instead, human experts had to hand-design rules and formulas – called 'features' – to describe things like edges, corners, or color patterns.



WHY CNNs CHANGED EVERYTHING

Instead of **humans hand-crafting features**, a CNN learns its own filters directly from data during training. This lets it automatically discover the best patterns to look for – something no hand-designed method could do at scale.





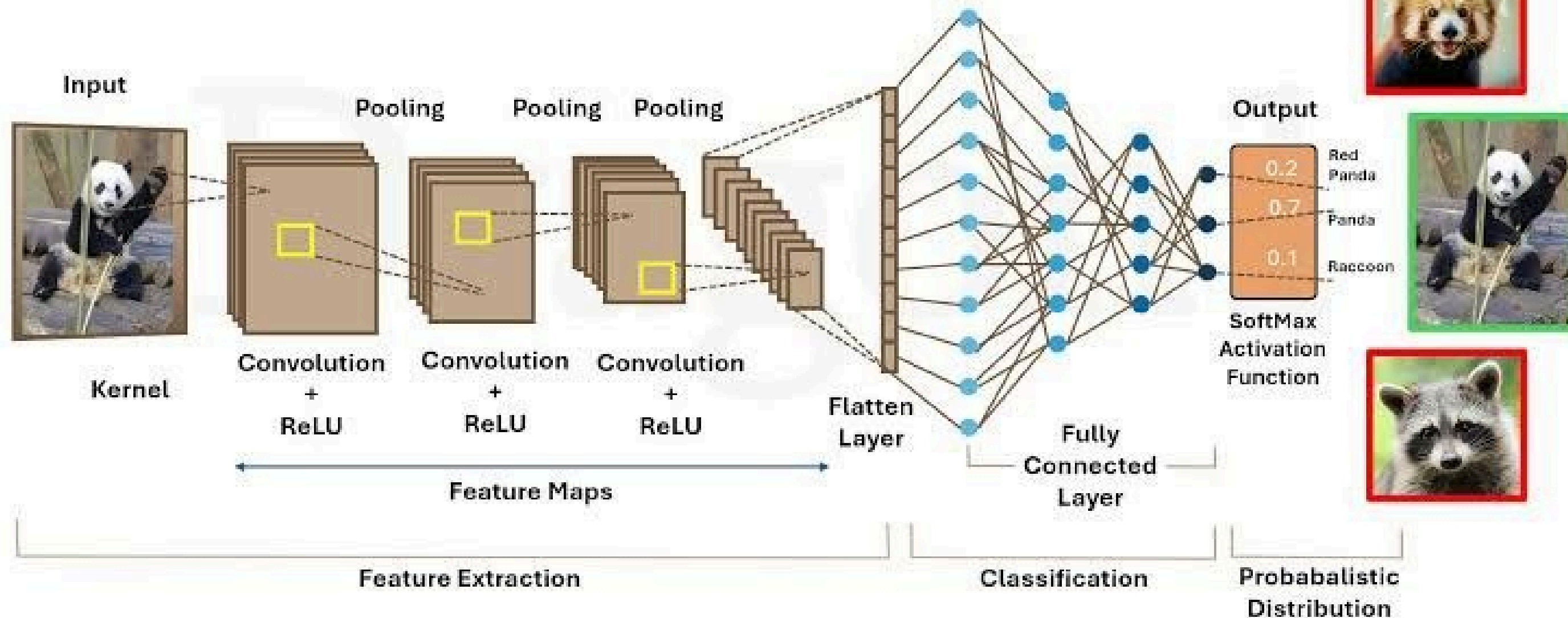
Section 2

Introduction to CNNs

INTRODUCTION

A **Convolutional Neural Network (CNN)** is a type of neural network built specifically to process grid-like data, such as images. It automatically learns to recognize patterns like edges, shapes, and objects – without being told what to look for

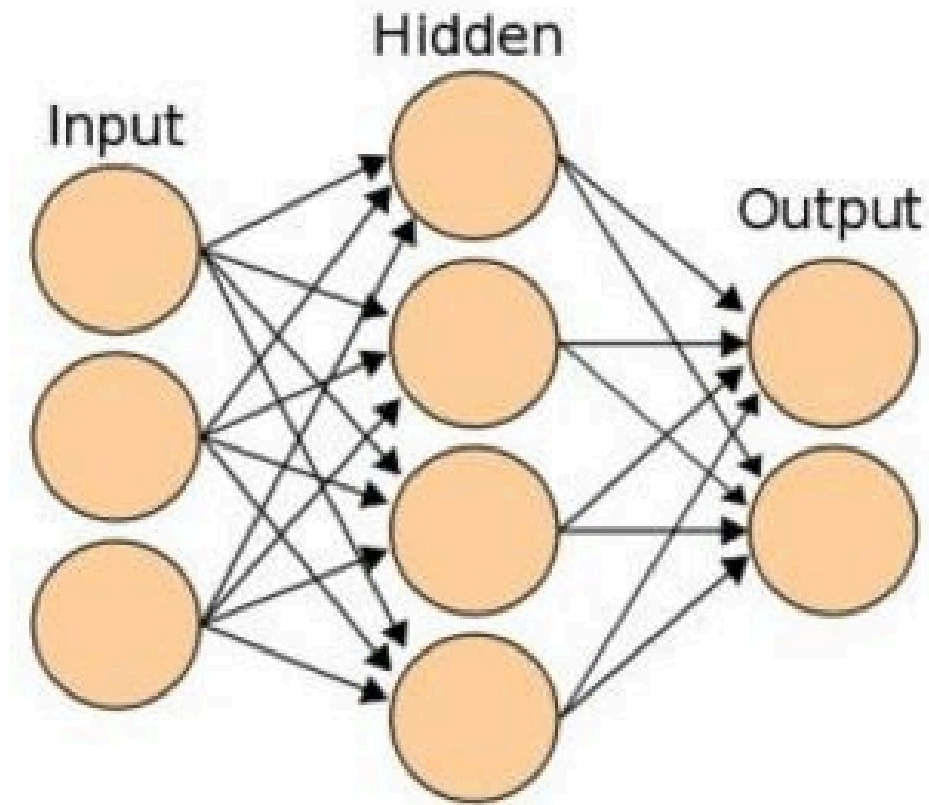
Convolutional Neural Networks (AI Deep Learning)



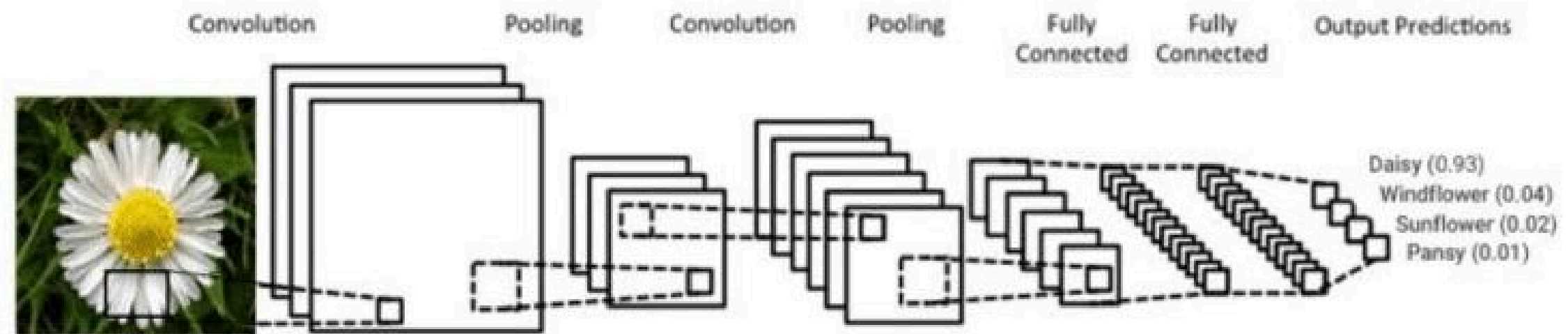
WHY CNNs

Images have spatial structure:

- nearby pixels are related to each other. CNNs preserve these spatial relationships, while a traditional network would flatten the image into a long list of numbers and lose that structure.

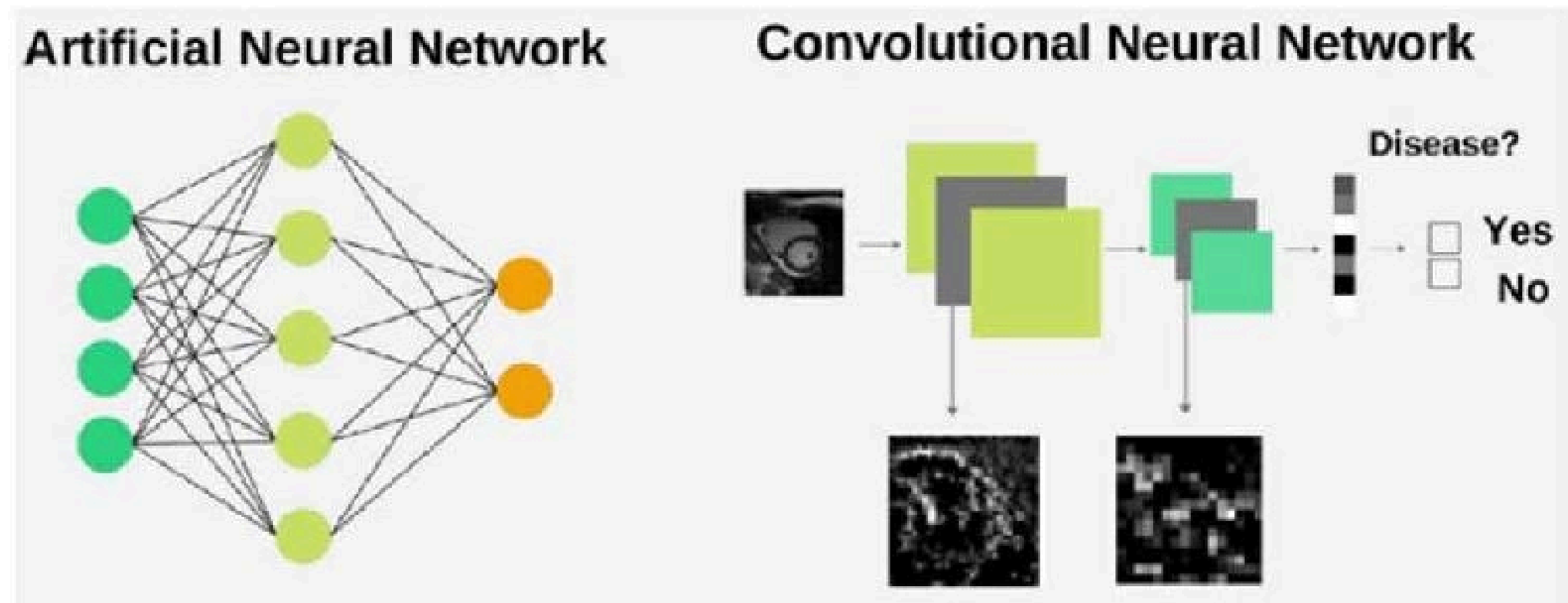


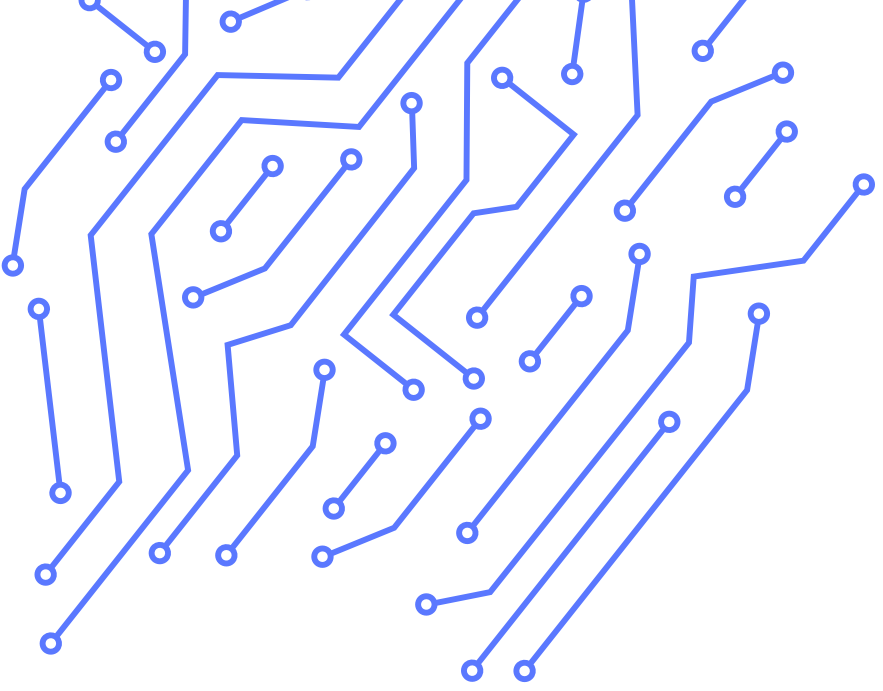
Convolutional Neural Network (CNN)



CNN VS. TRADITIONAL NEURAL NETWORKS

Traditional (fully connected) networks treat every pixel as a separate, independent input, which needs huge numbers of parameters and ignores spatial patterns. **CNNs** share the same small filter across the whole image, making them far more efficient and better at spotting a pattern no matter where it appears.





Section 3

Image Representation



PIXELS

A **pixel** is the smallest unit of a digital image – a single point holding a color or brightness value. An image is simply a grid made up of thousands of these pixels.

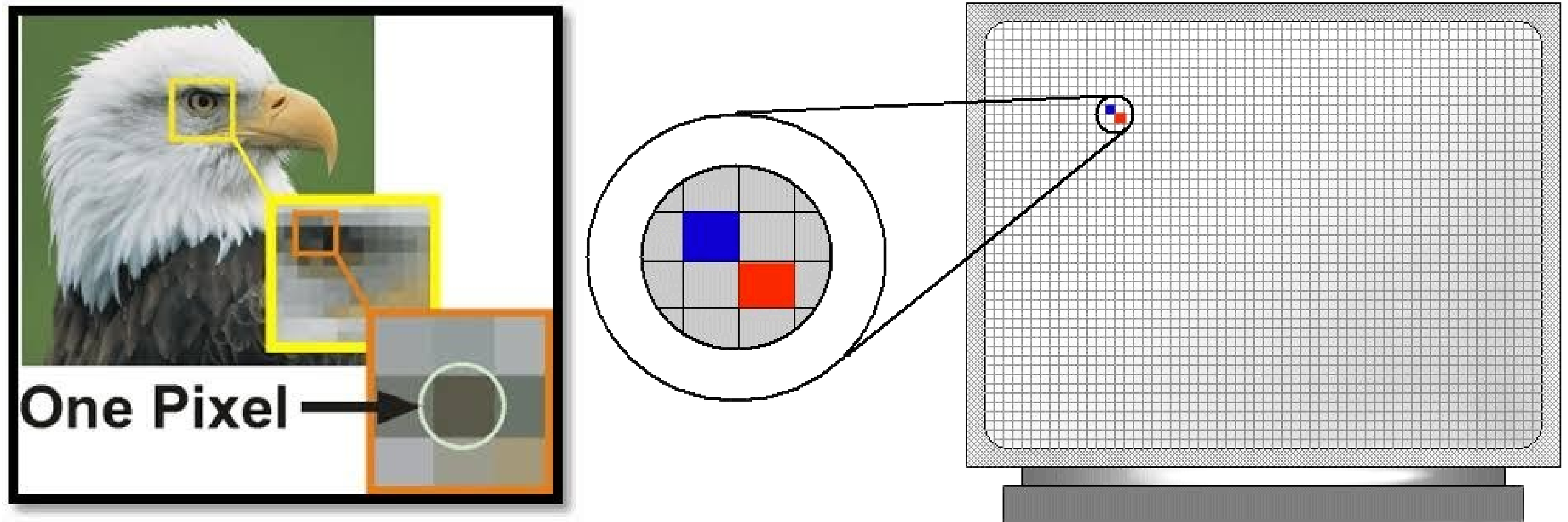
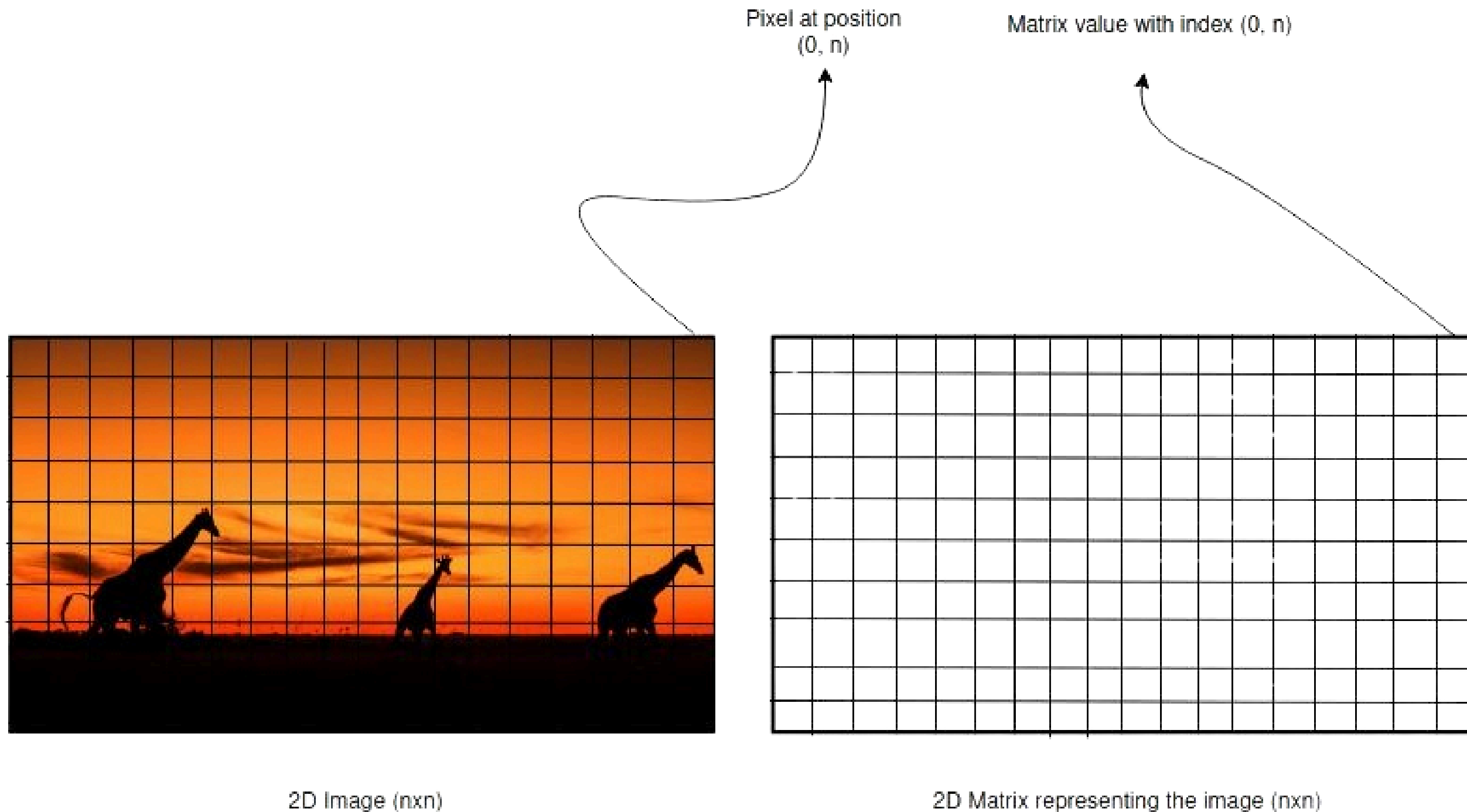


IMAGE DIMENSIONS

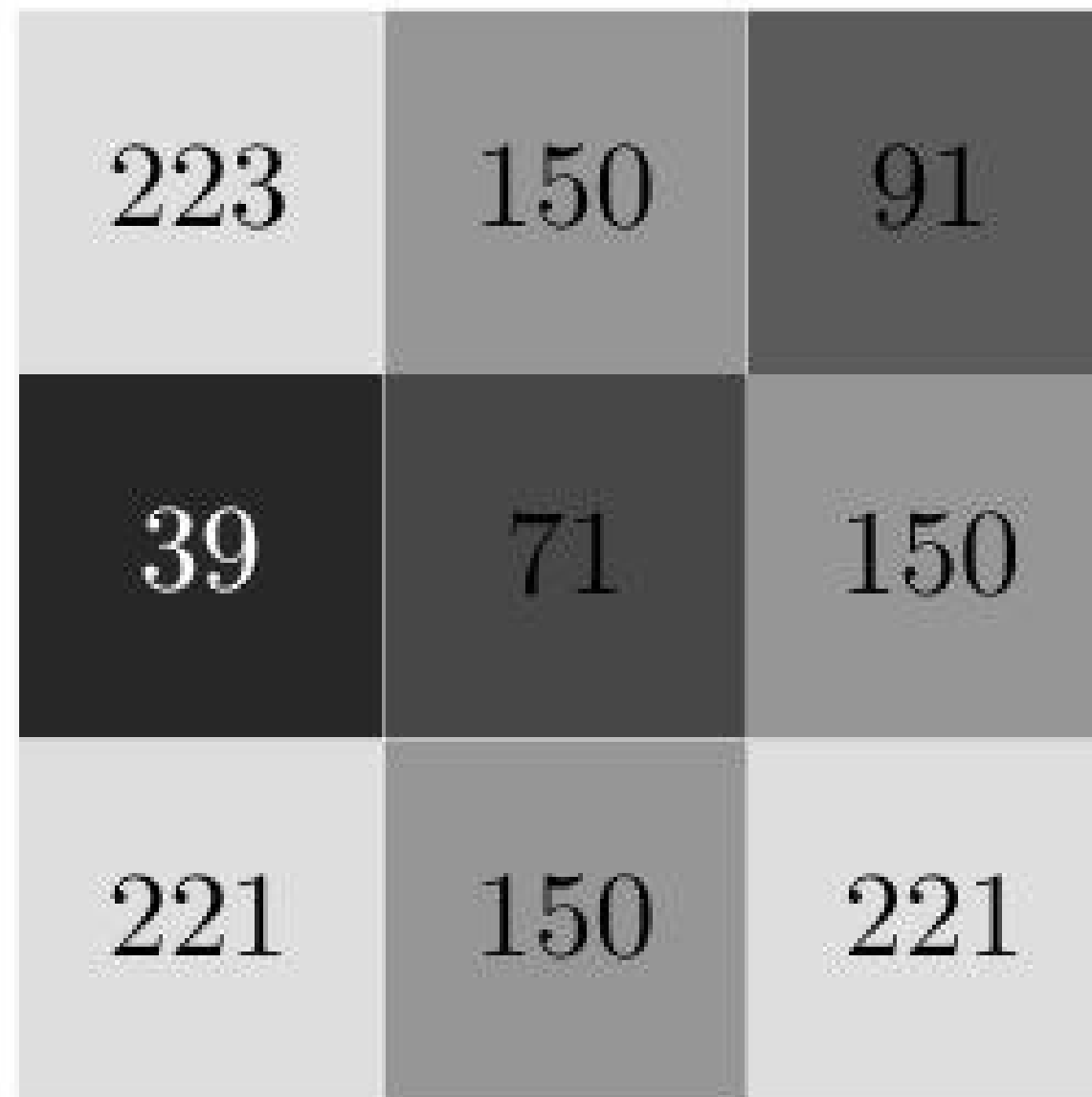
Every image has a Height and a Width, measured in pixels (for example, 224×224). These dimensions define the size of the grid the CNN will process.



CHANNELS (RGB / GRAYSCALE)

A **grayscale** image has 1 channel, storing only brightness. A **color** image has 3 channels – Red, Green, and Blue – which combine to form every color you see.

Grayscale



RGB

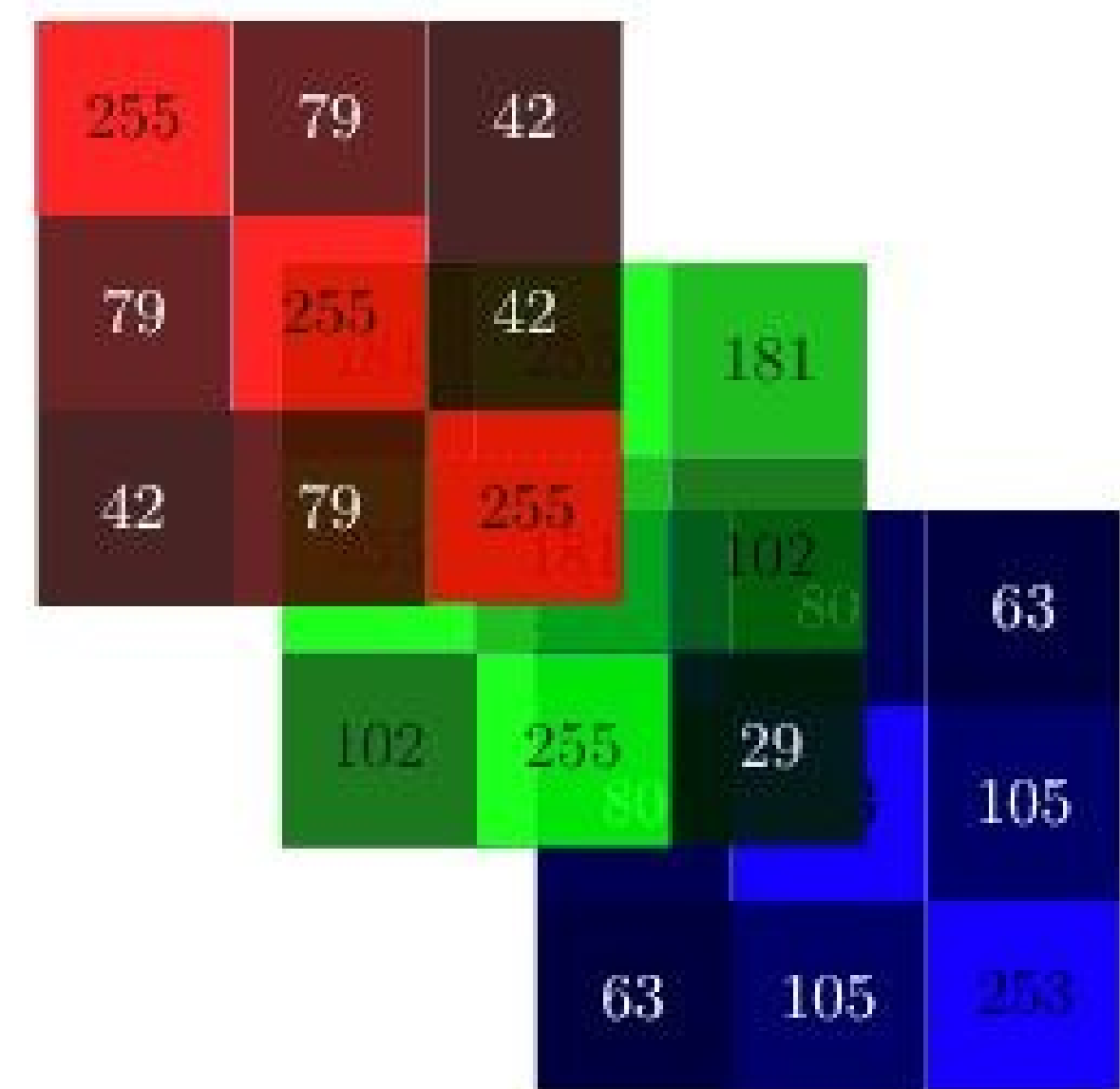
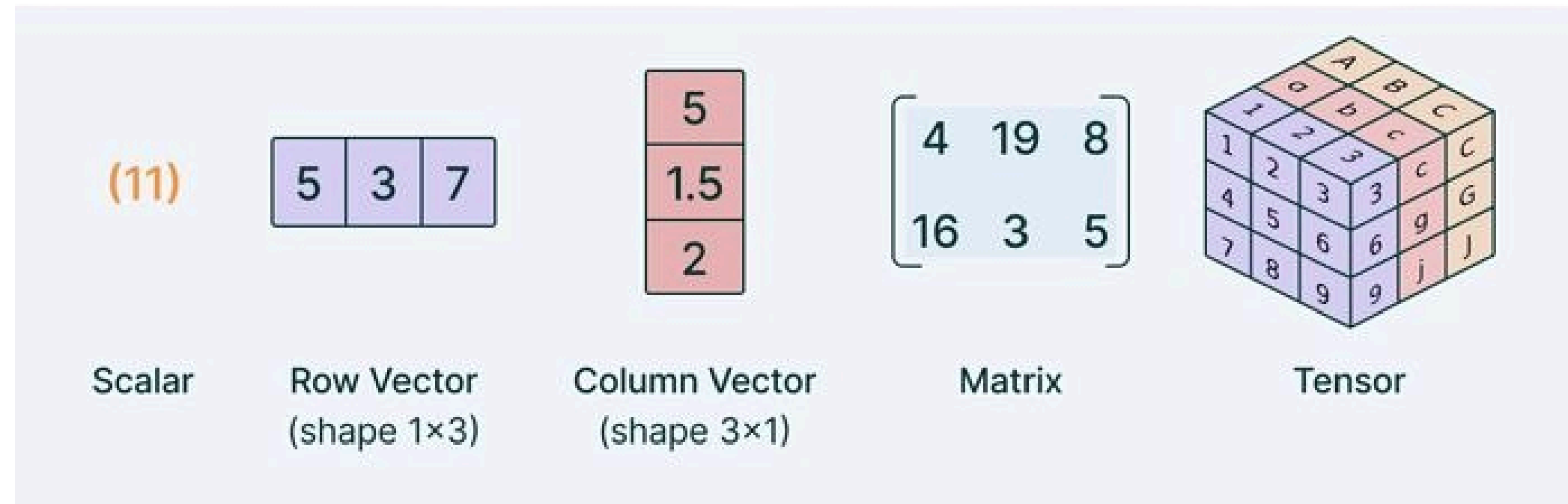
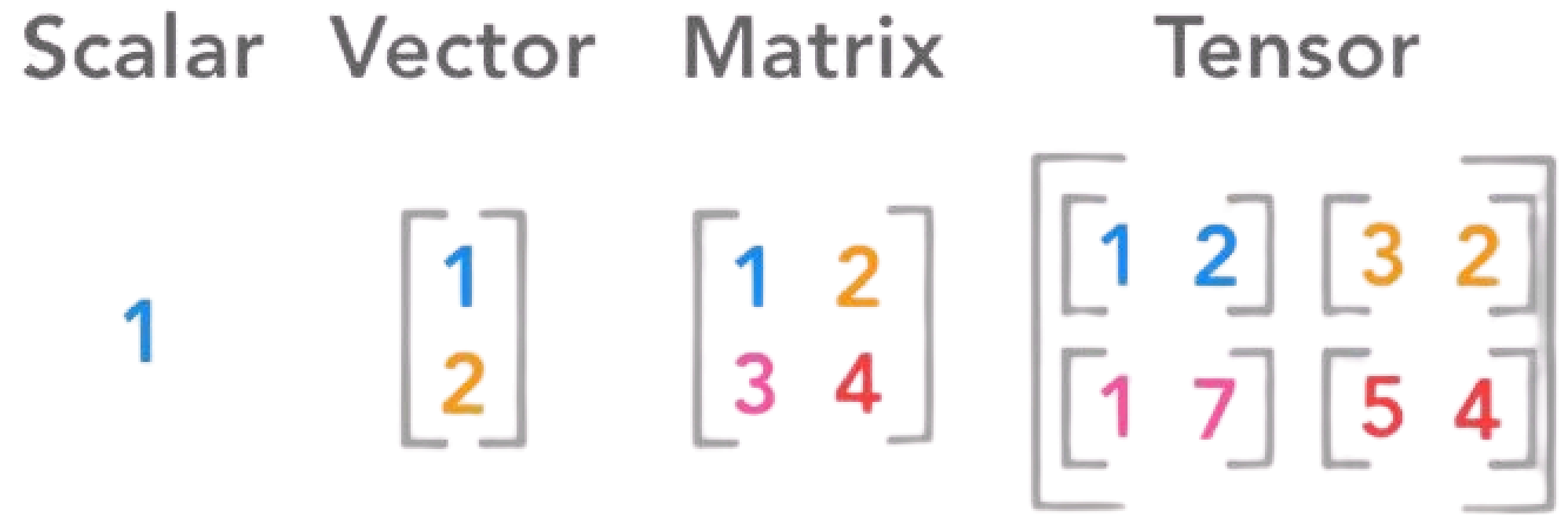
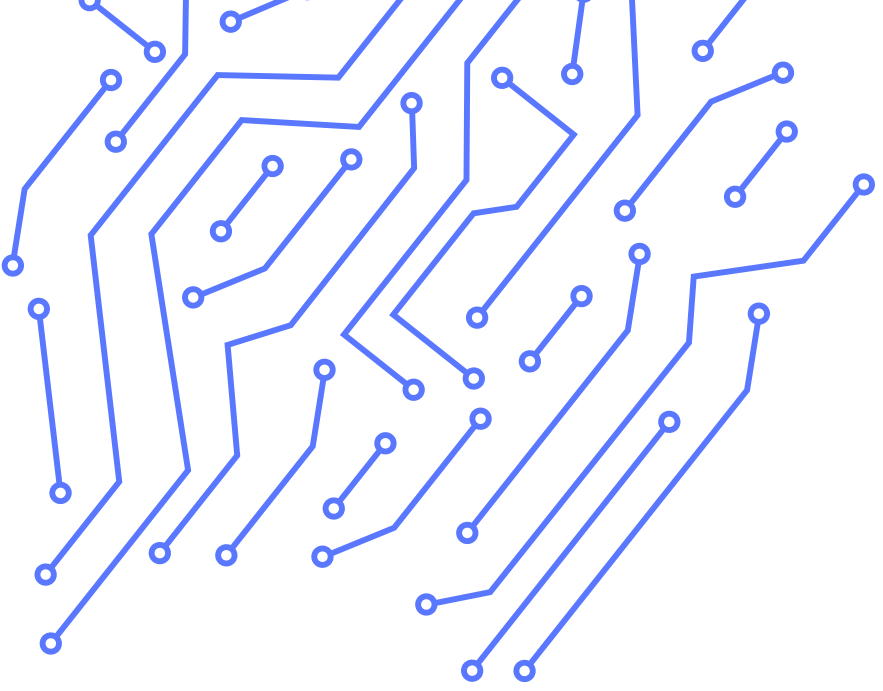


IMAGE TENSORS

A **CNN** doesn't see a 'picture' – it sees a 3D array of numbers shaped Height × Width × Channels. This numerical grid, called **a tensor**, is the actual input fed into the network.





Section 4

Convolution



INTRODUCTION

Convolution is the core operation of a CNN. A small filter slides across the image, multiplying its values with the pixels underneath and summing the result to detect a specific pattern at that location.

Input image

9	4	1	2	2
1	1	1	0	4
1	2	1	0	6
1	0	0	2	1
9	6	7	4	1

Filter

0	2	1
4	1	0
1	0	1

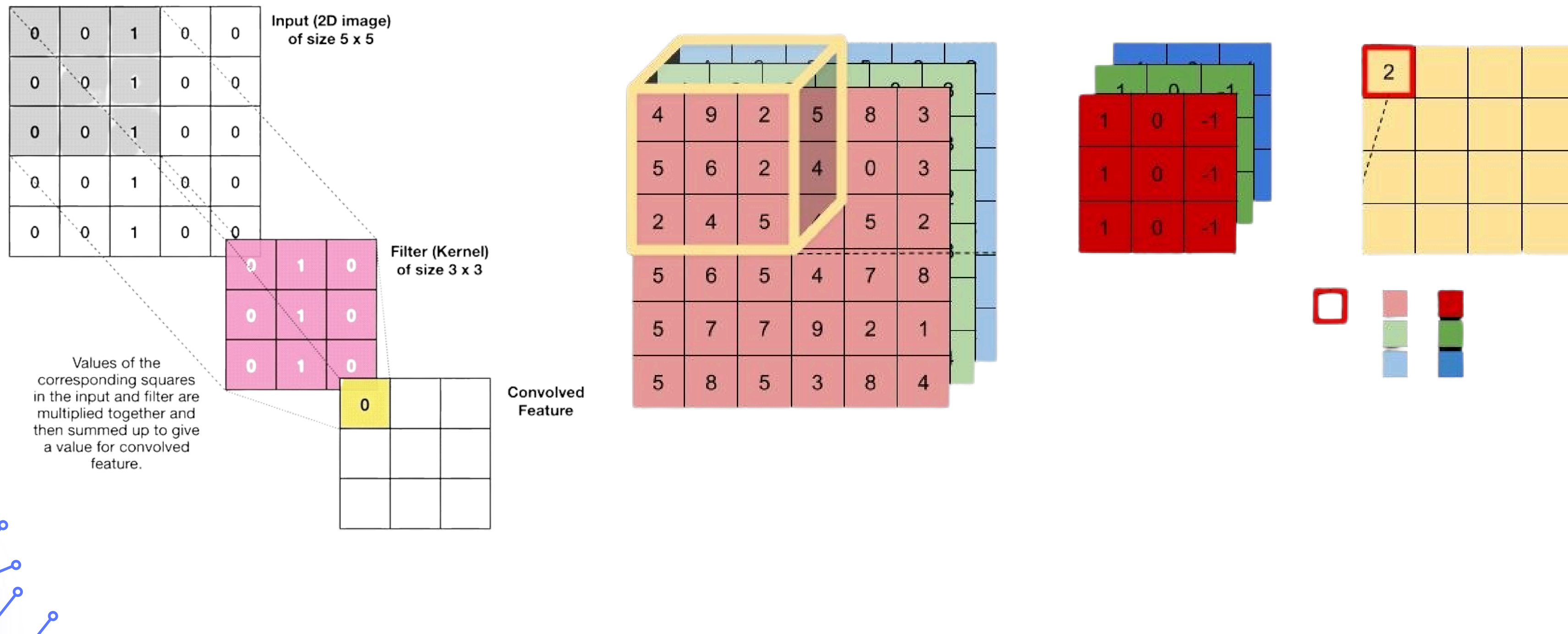
Output array

$$\begin{aligned}\text{Output}[0][0] &= (9*0) + (4*2) + (1*4) \\ &+ (1*1) + (1*0) + (1*1) + (2*0) + (1*1) \\ &= 0 + 8 + 1 + 4 + 1 + 0 + 1 + 0 + 1 \\ &= 16\end{aligned}$$



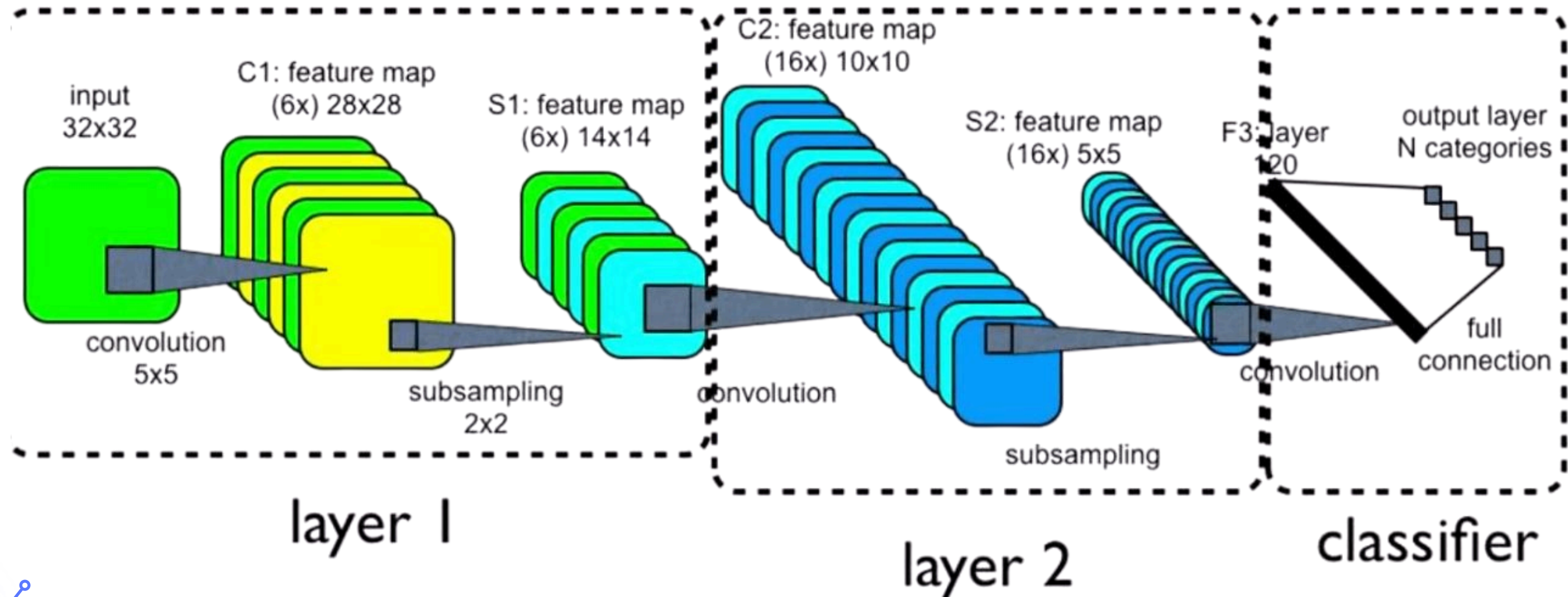
FILTERS / KERNELS

A filter (or kernel) is a small matrix of numbers – often 3×3 – that the network learns during training. Different filters learn to detect different features, such as edges, corners, or textures.



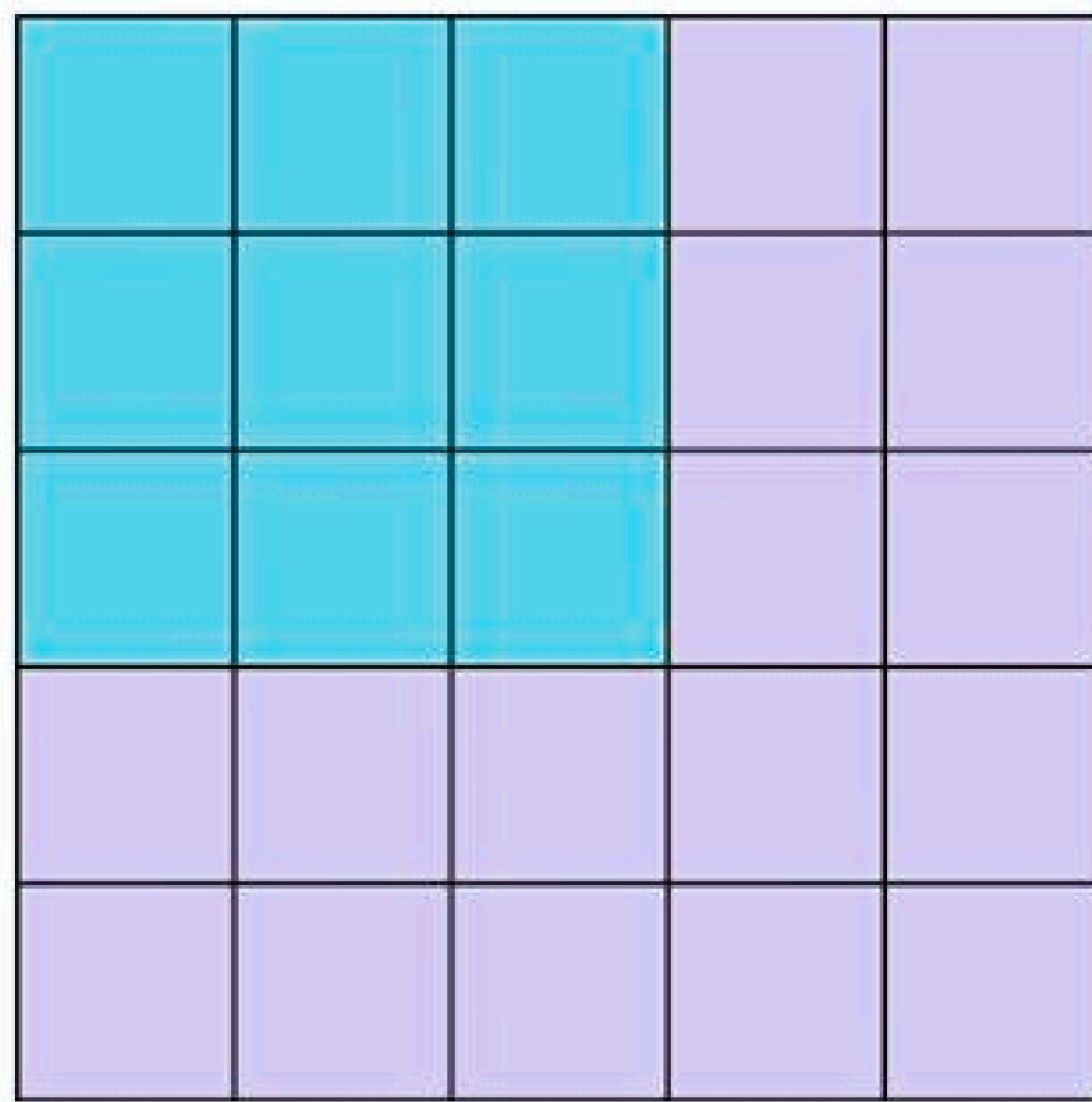
FEATURE MAPS

As a filter slides across the whole image, it produces a **feature map**: a new grid showing where in the image that particular pattern was found, and how strongly.

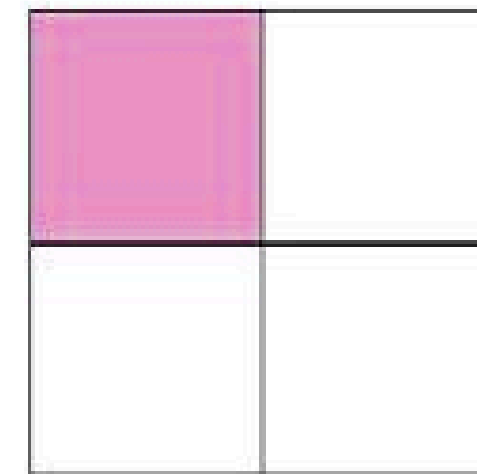


STRIDE

Stride is how many pixels the filter moves at each step. A stride of 1 moves one pixel at a time (more detail, bigger output); a larger stride skips more pixels (less detail, smaller output).



Stride 2

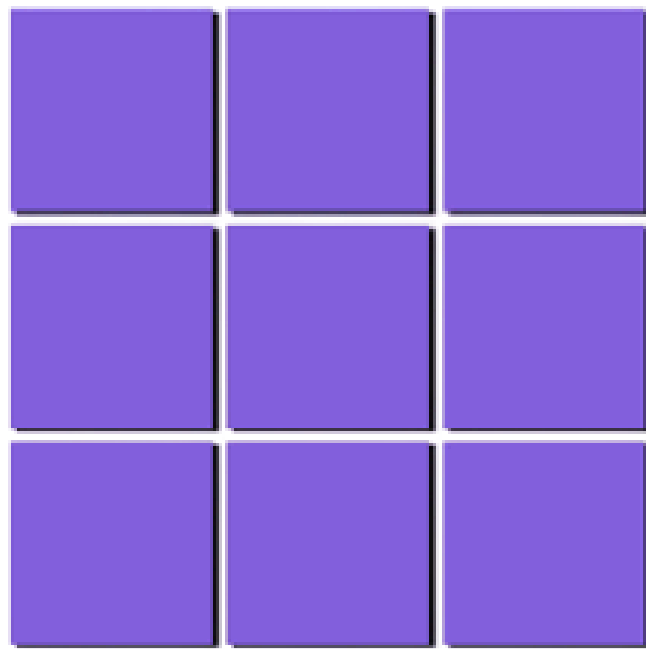


Feature Map



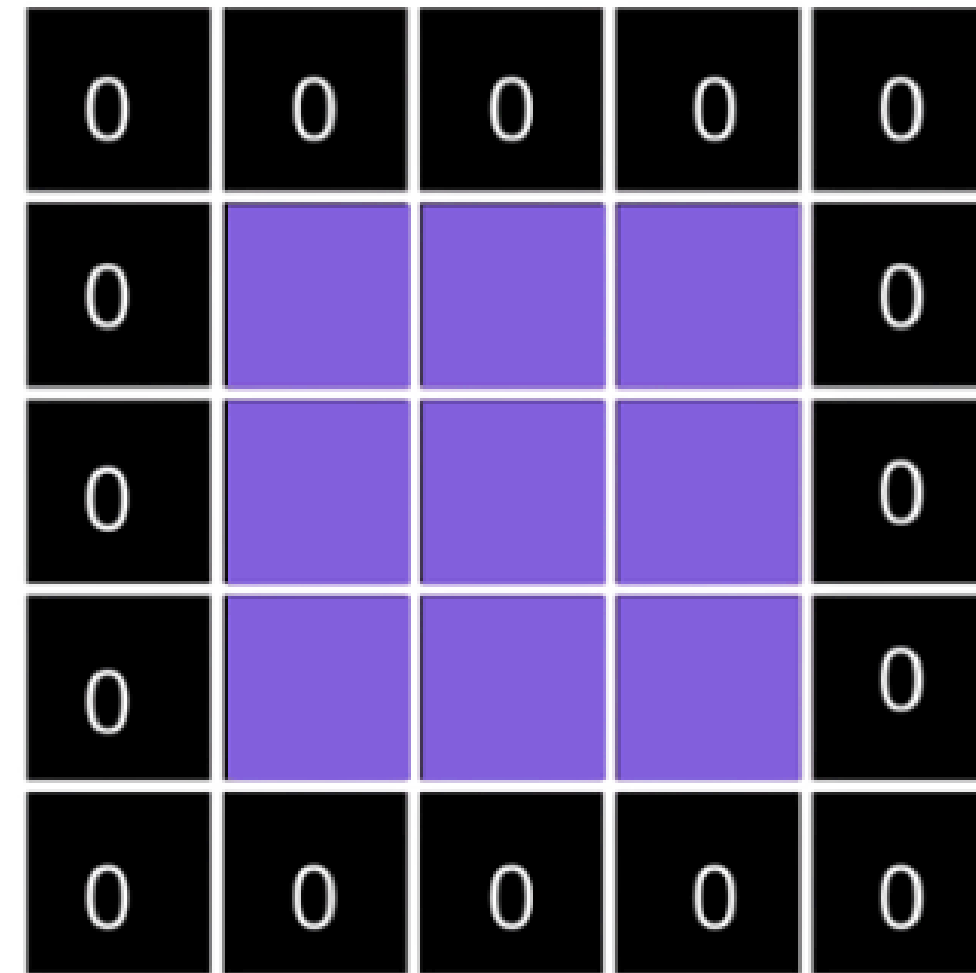
PADDING

Padding adds a border of zeros around the image before convolution. This lets the filter properly process edge pixels and gives control over the size of the output feature map.

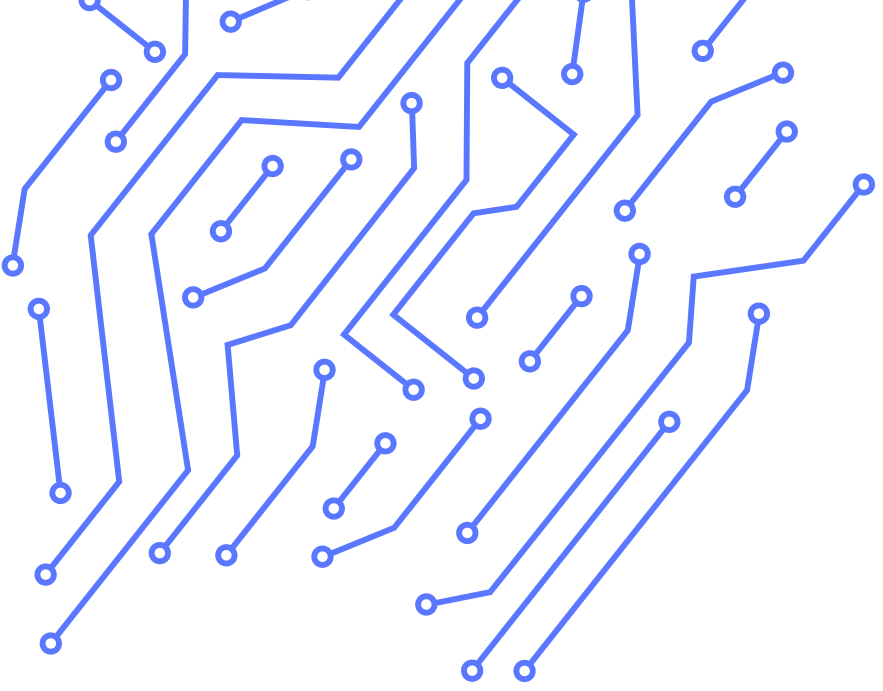


Input Image

Applying padding
of 1 on 3x3



Padded Image



Section 5

Pooling

INTRODUCTION

Pooling shrinks the size of feature maps while keeping the most important information. This reduces the amount of computation needed and helps the network focus on what matters.

Why pooling:

Pooling reduces the size and computation cost of the network, helps prevent overfitting, and makes the model more robust to small shifts or distortions in the image.



MAX POOLING

Max pooling looks at a small region of the feature map (e.g., 2×2) and keeps only the highest value – the strongest signal that a feature was detected there.

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Max Pool
→

Filter - (2×2)
Stride - $(2, 2)$

9	7
8	6

AVERAGE POOLING

Average pooling takes the average of the values in each region instead of the maximum, producing a smoother, more gradual downsampling of the feature map.

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Average Pool



4.25	4.25
4.25	3.5



THANK YOU

I hope you understand the
session well

Thanks for your attention